Implementation of Tarjan's Strongly Connected Components Algorithm

Connie Deng and Kyle Sullivan

BACKGROUND

Strongly Connected Components (or SCCs) can be defined as the maximal set of vertices, such that for any two vertices within a directed graph, there exists a path from one vertex to another and a path back. This overall structure allows SCCs to capture cycles / connected regions, which can be modelled as circular dependencies. In terms of practical applications, finding strongly connected components is important in the context of operating systems, where deadlocks can be represented as a SCC and must be efficiently detected. In another example, dependency management benefits from quickly finding circular dependencies, which is another application of finding SCCs. Since identifying strongly connected components within a graph is a fundamental problem faced in different computing fields, there have since been multiple algorithms that aim to address this problem, such as Tarjan's algorithm.

Robert Tarjan is an accomplished computer scientist responsible for the development of an algorithm that finds strongly connected components, along with a plethora of other contributions. Tarjan obtained a Ph.D. in Computer Science from Standard University in 1972. His thesis was based on the development, alongside John Hopcroft, of the first linear time algorithm for the planarity of a graph. That is, whether a graph can be drawn on a plane such that no edges cross. His Ph.D. was completed under the supervision of Robert W. Floyd, of the Floyd-Warshall Algorithm. In his thesis, Tarjan argued for the use of Depth First Search and the use of adjacency lists rather than adjacency matrices on sparse graphs. This would lead to the development of Tarjan's Strongly Connected Components algorithm, which is also a DFS based, linear time algorithm. Tarjan has several other noteworthy contributions to computer science, including being the first to show that Union-Find operations could be achieved in constant time, creation of data structures like the dynamic tree, the splay tree, and the fibonacci heap, and creating the first linear time minimum spanning tree algorithm (King).

Upon publishing his algorithm in June 1972, Tarjan writes that previous approaches to this problem had been of time complexity $O(|V|^2)$ and $O(|V| \log |V|)$. However, the shortcoming of previous approaches was that discovered strongly connected components could be a part of a larger component, so those algorithms would have to check for components more than a linear number of times (Tarjan, p. 155). So, Tarjan suggests a new algorithm that overcomes this issue, and achieves a runtime of $O(|V|+|E|)$.

PROBLEM STATEMENT

In our paper, we were especially interested in understanding how Tarjan's algorithm worked mechanically, to which we will discuss our implementation later on. We felt it was necessary to identify key differences between how Tarjan's algorithm operated compared to other strongly connected component finding algorithms (i.e. identify complexities, trade offs, etc.).

ALGORITHM DESCRIPTION

Tarjan's Strongly Connected Component algorithm is an algorithm that produces the SCCs of a directed graph. In his publication about the algorithm, Tarjan starts by describing the relevant edges to a strongly connected component. He writes that tree edges, forward edges, which he calls *fronds*, and cross edges, denoted as *cross-links*, are relevant; the encountered back edges are not used in this algorithm. By using the Depth-First Search algorithm, a spanning forest can be created, along with sets of fronds and cross-links. Tarjan says that if nodes v and w are both in the spanning tree, then they must share an ancestor. Moreover, if the node u is the greatest numbered ancestor of v and w, where the number is time of discovery during DFS, then u is in the same strong component of v and w. Then, Tarjan writes, finding strongly connected

components reduces to the problem of finding the roots of strongly connected components. The core of his algorithm is the following equation.

$$\text{LOWLINK}(v) = \min(\{v\} \cup \{w | v \xrightarrow{*} \cdots \rightarrow w \;\&\; \exists u (u \xrightarrow{*} v \;\&\; u \xrightarrow{*} w \;\&\; u \text{ and } w \text{ are in the same strongly connected component of } G)\})$$

LOWLINK (v) is the minimum discovery value of the node v such that there is a path from v to node w where there is a cross-link or frond in the path, and there is a node u such that there is a path from u to v, and a path from u to w, and u is in the same strongly connected component of v and w (Tarjan, pp. 155-156). Based on the pseudocode published by Tarjan, we implemented the following algorithm in C++:

```
vector<vector<int>> graph; //[i][j] where i is node num,
and j is whether node is adjacent
vector<int> discoveryTime;
vector<int> LOWLINK;
vector<bool> isOnStack; //for quick a quick look
stack<int> TarjanStack;
int index = 1;
vector<vector<int>> SCCResults;

//DFS FUNCTION
void strongConnect(int v) {
    // Set the depth index for v to the smallest unused
index
    discoveryTime[v] = index;
    LOWLINK[v] = index;
    index = index + 1;
    TarjanStack.push(v);
    isOnStack[v] = 1;

    //Consider successors of v
    for(size_t w = 0; w < graph[v].size(); w++){//for (int
w : graph[v]) {
     if(graph[v][w]){
          if(!discoveryTime[w]){ //then v shares an edge
with an undiscovered node
               strongConnect(w);
               LOWLINK[v] = min(LOWLINK[v],LOWLINK[w]);
          }
          else if(discoveryTime[w] < discoveryTime[v]){
//then v to w is a back edge or a cross edge
               if(isOnStack[w])
                    LOWLINK[v] = min(LOWLINK[v],
discoveryTime[w]);
          }
     }
    }
```

```
    // If v is a root node, pop the stack and generate an
SCC
    if (LOWLINK[v] == discoveryTime[v]) {
        vector<int> SCC;

     while(!TarjanStack.empty() &&
discoveryTime[TarjanStack.top()] >= discoveryTime[v]){
          SCC.push_back(TarjanStack.top());
          isOnStack[TarjanStack.top()] = false;
          TarjanStack.pop();
     }
     int currSCCResultRow = SCCResults.size();
     SCCResults.push_back(vector<int>());
     for(size_t i = 0; i < SCC.size(); i++){
          SCCResults[currSCCResultRow].push_back(SCC[i]);
     }
    }
}
```

We start by declaring several data structures. Firstly, we used an adjacency matrix to store an inputted graph. Next, we used vectors that store the discovery time and LOWLINK value of each node. A stack was also implemented that stores each node when discovered via Depth First Search. This stack, which we call Tarjan Stack, is used to keep track of discovered nodes that may be in the same component. There is also an array that stores whether each node is on the stack, called isOnStack. We also initialize a variable index, which keeps track of the discovery time of the DFS. Finally, we define a vector of vectors, SCCResults, that is used to store each distinct component and their elements. Next we defined the StrongConnect(v) function. Given a node v, StrongConnect performs a DFS, while adding elements to their components. At the start of a StrongConnect call, the discoveryTime[v] and LOWLINK[v] are set to the current discovery time, and the discovery time is incremented. Since, from the current node's point-of-view, the reachable node with the smallest discovery time is itself, LOWLINK[v] = index = discoveryTime[v]. Additionally, the current node is placed on TarjanStack. Next, in regular DFS fashion, the algorithm explores paths that originate from v. For each adjacent node, their edge type is analyzed, and different steps are taken depending on the type.

If an adjacent node, w, was not discovered before accessing it via the directed edge from v to w, it is a tree edge. In this case, STRONGCONNECT(w) is recursively called, and after terminating, LOWLINK[v] is set to the minimum of LOWLINK[v] and LOWLINK[w]. This is done because it is possible that while exploring paths originating from w, a path may have been found that led to a node with a smaller LOWLINK value than LOWLINK[v]. Thus, LOWLINK[v] is updated to the reachable node with the smallest discovery time. However, if the node w was already discovered before traversing the edge from v to w, the two nodes share a back or cross edge. In this case, the algorithm checks if node w is on the stack. If not, node w is not a candidate for the strongly connected component that v is a member of, so it is ignored. If on the stack, LOWLINK[v] is set to the minimum between LOWLINK[v] and discoveryTime[w]. The logic behind this is if v and w are candidates for the same component, and discoveryTime[w] is less than LOWLINK[v], a member of the same component with a smaller discovery time than LOWLINK[v] has been discovered. So, the algorithm may update LOWLINK[v] to account for this discovery. These two cases for the different edge types are

repeated until all adjacent nodes have been completed for all edges originating from v. Next, the algorithm determines if a component must be built.

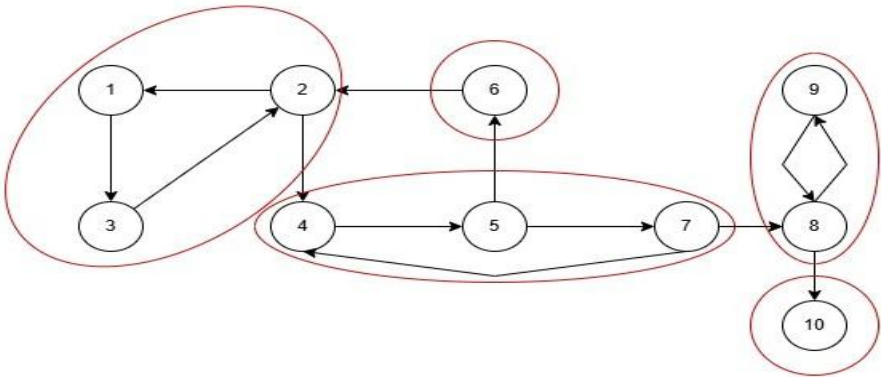
After completing the edges of node v, and while still at node v, the algorithm checks to see if v is the head of a component. In the code, this is achieved by comparing the values of discoveryTime[v] and LOWLINK[v]. If they are not equal, node v is not a head, so the algorithm simply completes node v and returns from the recursive call StrongConnect(v). However, if they are equal, v is the head of a component. The reasoning behind this is if the LOWLINK[v] equals discoveryTime[v], there is no reachable node that has a smaller discovery time than itself. Thus, v is the oldest node in its component, and since all nodes with a smaller discovery time have been discovered, and since all nodes of the component are higher in the stack than the head, a component is ready to be rebuilt. The algorithm then enters a while loop to check if the node on top of TarjanStack has the same LOWLINK value as node v. If so, the node is popped from the stack, its position in isOnStack is set to false, and the node is then added to the current component, which is represented as a vector, SCC. This loop continues until there is a node on top of the stack with a LOWLINK value different from LOWLINK[v], or the stack is empty. In the case where there are no nodes on the stack which share the same LOWLINK value with v, v is the head of a component only containing itself, even if it does not have a self-edge. At the point of termination of the while loop, the component has been built, so vector SCC is added to the vector of vectors SCCResults, and node v is now complete, so the algorithm returns from the recursive call StrongConnect(v). Using our implementation of Tarjan's algorithm, we were able to produce strongly connected components on several different graphs.

RESULTS & DISCUSSION

RESULTS:

To test our implementation, we tried Tarjan's algorithm on several graphs. Firstly, we wished to test different scenarios of node interactions. As seen in Figure 1, the red circled sections of nodes are the expected SSCs produced by our implementation.
nodes 1, 2, and 3 can all reach each other, and only themselves; nodes 4, 5, and 7 contain directed edges that go outside of their components; nodes 6 and 10 are both in their own components, where 6 has an edge to another node outside its component, and 10 does not; and nodes 8 and 9 where both nodes in the component are adjacent.

Figure 1

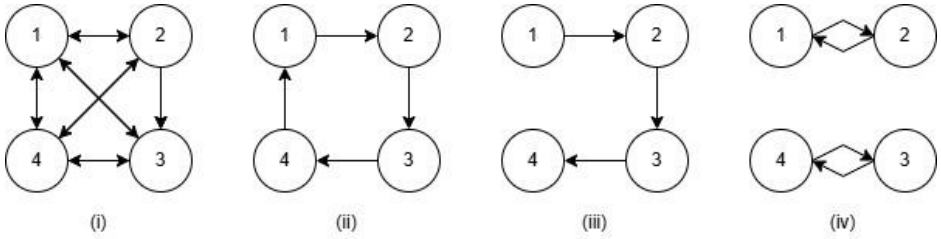


Running our algorithm on Figure 1 produced the following results:

```
1 3 2
6
8 9
4 5 7
10
Runtime: 2390 nanoseconds
```

As can be seen in the output, each component is placed on a new line, and the total runtime of the algorithm on the graph is shown in nanoseconds. For Figure 1, the output matches the expected SCCs in the graph. We also tested the algorithm on several simple graphs, to ensure correct behavior.

Figure 2



Of the graphs shown in Figure 2, the first is a complete graph and the second is a connected graph. In both cases, we expected there to be a single component containing all nodes in the graphs. The algorithm outputted the following results:

```
1 2 3 4
Runtime: 440 nanoseconds

1 2 3 4
Runtime: 320 nanoseconds
```

So, in these cases, our implementation produced the correct output. Next, we wanted to ensure that nodes that do not form cycles with other nodes remain in their own isolated components. To test this case, we developed the 3rd graph in figure 3. We expected this graph to contain four different components, each only containing a single node. Tarjan's algorithm produced this output:

```
4
3
2
1
Runtime: 400 nanoseconds
```

Once again, our output matched our expectations. Finally, we wanted to ensure that our implementation worked correctly on a disconnected graph, so we designed the fourth and final node of Figure 2, and tested our implementation on it. We expected that the algorithm would produce two components: one containing nodes 1 and 2, and another containing 3 and 4. The algorithm outputted the following results:

```
1 2
3 4
Runtime: 390 nanoseconds
```

In all four cases of Figure 2, the SCCs produced by Tarjan's algorithm matched our expectations. In addition to the graphs shown, we introduced self-edges to the nodes in the third and fourth graphs of Figure 2, and we found that the produced components remained the same. In terms of complexity, Tarjan has shown that his algorithm, and by extension our implementation, has a time complexity of $O(|V| + |E|)$. Additionally, since we used an adjacency matrix, our algorithm has a space complexity of $(|V|^2)$. Using adjacency lists instead of could reduce this complexity to $O(|V| + |E|)$.

COMPARISON DISCUSSION

Tarjan's algorithm is one of the most well-known strongly connected components finding algorithms, but there are other specific implementations. These algorithms being: Kosaraju's, a Path Based Strong Component Algorithm, a divide and conquer approach, and brute force.

I) Kosaraju's Algorithm

Kosaraju's Algorithm is most likely the second most popular SCC finding algorithm, and is also a DFS based algorithm. It works by using two depth first searches, where the first one sorts the nodes by its finishing time in the original graph, while the second DFS is responsible for actually identifying the strongly connected components. The second DFS reverses all the edges in the graph, and nodes are processed in reversed order according to finishing time. Kosaraju's algorithm runs in $O(|V| + |E|)$ time, and is slightly less efficient than Tarjan's.

II) Path Based Strong Component Algorithm

Several noteworthy computer scientists have developed different forms of this algorithm, including Ian Munro, responsible for optimal binary search trees, priority queues, and hashing, along with Edsger Dijkstra, of Dijkstra's Algorithm. This algorithm is quite similar to Tarjan's algorithm. Both are DFS based algorithms, and they follow the same sequence of steps for searching the graph and analyzing edges. Where they differ is their use of data structures. While Tarjan's algorithm uses LOWLINK to keep track of the minimum reachable node, and how to determine which nodes are heads of components, the Path Based algorithm uses an additional stack compared to Tarjan's. Thus, this algorithm uses three stacks: stack S to keep track of nodes not assigned to a component, in order of DFS discovery, stack P to contain the encountered nodes that are not known to belong to different components, along with the recursive DFS stack. The Path Based approach, like Tarjan's, has a linear time and space complexity of $O(|V|+|E|)$ (Minea, pp. 13-15).

III) Divide-and-Conquer Approach

Tarjan's algorithm is ultimately rooted in depth first search, and hence, also inherits the same sequential issues, which makes Tarjan's difficult to parallelize and prevents Tarjan's algorithm from being as efficient on large, parallel systems. In a paper by Coppersmith et al. from IBM research, this specific limitation of Tarjan's is acknowledged, and proposes a Divide and Conquer approach (formally named Divide and Conquer Strong Components) to address those bottlenecks. This algorithm first selects a pivot node v, and then computes two specific sets. The first set is the set of nodes reachable from v, and the second set is the set of nodes that can reach to v. The intersection of these two sets is able to identify the strongly connected component that v belongs to. This component is then removed from the set, with a guarantee that this specific component can't show up in future intersections. The algorithm works recursively, and ultimately allows the overall problem to be broken up and solved in parallel. This specifically provides a time complexity of $O(V \log E)$, and controlled recursion depth, while randomized pivots prevent the same node from being visited too many times.

IV) Brute Force

The essence of the brute force approach is to calculate a matrix of all the directed edges in a graph, then build the components from the matrix. Creating this matrix is done using algorithms like boolean matrix multiplication or the Floyd-Warshall algorithm (Miller & Sutner, p. 11), both of time complexity $O(|V|^3)$ and space complexity $O(|V|^2)$. Using the matrix, for each pair of nodes i and j, if B[i][j] = 1 and B[j][i] = 1, add i and j to the same component. This takes an additional $O(|V|^2)$ steps. Thus, the brute force approach is expensive, compared to Tarjan's algorithm in time and space, and is not a worthwhile approach.

CONCLUSION:

Overall, this paper examines and implements Tarjan's Strongly Connected Component algorithm, and investigates its counterparts and its uses. Implementing the algorithm allowed us to understand its overall mechanics, and how discovery times and LOWLINK values are used, and how its overall depth first search traversal approach is efficiently identifying strongly used components. Testing our implementation of Tarjan's algorithm on various graph inputs has confirmed its ability to recognize strongly connected components in different circumstances, such as in cyclic, acyclic, and disconnected graphs. We also researched practical applications of identifying SCCs, such as deadlock detection in operating systems and package cycle detection in dependency management, indicating the need for different algorithms for finding SCCs. While comparing Tarjan's SCC algorithm to other approaches such as Kosaraju's, path based methods, divide and conquer, and brute force, we found that it operates very efficiently overall, with some noticeable drop offs in performance when it encounters large parallel graphs. Despite these limitations, Tarjan's algorithm is still one of the most well known SCC detection algorithms, and still has numerous applications within computer science.

References

A. Neeraja, C. M. Begum, K. V. Reddy, R. H. Koushal and D. K. V, "An Enhanced Deadlock Detection Methodology Using Tarjan's Algorithm," 2024 8th International Conference on Computational System and Information Technology for Sustainable Solutions doi: 10.1109/CSITSS64042.2024.10816916

Coppersmith, Don; Fleischer, Lisa; Hendrickson, Bruce & Pinar, Ali. A divide-and-conquer algorithm for identifying strongly connectedcomponents, report, March 27, 2003; Berkeley, California. (https://digital.library.unt.edu/ark:/67531/metadc884594/: accessed December 13, 2025), University of North Texas Libraries, UNT Digital Library, https://digital.library.unt.edu; crediting UNT Libraries Government Documents Department.

Fleischer, L.K., Hendrickson, B., Pınar, A. (2000). On Identifying Strongly Connected Components in Parallel. In: Rolim, J. (eds) Parallel and Distributed Processing. IPDPS 2000. Lecture Notes in Computer Science, vol 1800. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-45591-4_68

King, V. (1986). Robert (Bob) Endre Tarjan. https://amturing.acm.org/award_winners/tarjan_1092048.cfm

Miller, G. & Sutner, K. (9/24/2020). *451: Strongly Connected Components* [PowerPoint presentation]. Carnegie Mellon University. https://www.cs.cmu.edu/~15451-f20/LectureNotes/dfs-scc.pdf.

Minea, M. (2/2/2021). *Lecture 1: Algorithms for Strongly Connected Components* [PowerPoint presentation]. University of Massachusetts Amherst. https://people.cs.umass.edu/~marius/class/h311-sp21/lec1.pdf.

Strongly Connected Components. GeeksforGeeks, www.geeksforgeeks.org/dsa/strongly-connected-components/. Accessed 13 Dec. 2025.

Tarjan, R. (1972, June). Depth-First Search and Linear Graph Algorithms. *SIAM Journal on Computing, 1*(2), 146-160. https://web.archive.org/web/20170829214726/http://www.cs.ucsb.edu/~gilbert/cs240a/old/cs240aSpr2011/slides/TarjanDFS.pdf